PRESIDENCE DE LA REPUBLIQUE

MINISTERE DE LA PLANIFICATION DU DEVELOPPEMENT ET DE LA COOPERATION



Institut Africain d'Informatique

Etablissement Inter-Etats d'Enseignement Supérieur
Représentation du Togo (IAI-TOGO)
07 **BP** 12456 Lomé 07 **Tél.** : (+228) 22 20 47 00
E-mail : iaitogo@iai-togo.tg **Site Web**: www.iai-togo.tg

Support de cours

Initiation au langage Pascal

Chargé de cours : M. AKOMAGBE Innocent

01000011 01101111 01101110 01100111 01110010 01100001
01110100 01110101 01101100 01100001 01110100 01101001
01101111 01101110 00100000 01100111 01100101 01100101

Année Académique 2024 - 2025

SOMMAIRE

Chapitre 1 : Outils et structure générale d'un programme pascal 2
I. Outils 2
II. Structure générale d'un programme en Pascal 3
Chapitre 2 : Les variables et les constantes 5
I. Les Types de variable 5
II. Déclaration de variable et de constante 5
Chapitre 3 : Les entrées et les sorties 8
I. Les sorties 8
II. Les entrées 8
III. Affectation d'une variable 9
Chapitre 4 : Opérateurs élémentaires 10
I. Opérateurs et fonctions arithmétiques 10
II. Opérateurs relationnels 10
III. Opérateurs logiques 11
IV. Opérateurs de bit à bit 11
V. Opérateur sur les chaines de caractères 12
VI. Quelques opérations avec les caractères 12
Chapitre 5 : Les Types personnalisés 13
I. Type Simple 13
II. Type structuré 13
III. Type intervalle 14
IV. Type énuméré 15
Chapitre 6 : Les instructions conditionnelles 18
I. Instructions 18
Chapitre 7 : Boucles itératives 21
I. Boucle FOR 21
II. Boucle conditionnelle 21
Chapitre 8 : Les Tableaux 24
I. Déclaration de tableaux 24
II. Créer un type "tableau" 25
III. Les tableaux dynamiques 25
IV. Quelques opérations sur les tableaux 26
Chapitre 9 : Fonctions et Procédures 28
I. Notions de sous programmes 28
II. Les procédures 28
III. Les fonctions 31
IV. Procédures et fonctions prédéfinies 32
Chapitre 10 : Les fichiers 35
I. Déclaration 35
II. Les fichiers textes (Text) 35
III. Les fichiers types (File of) 35
IV. Les fichiers simples (File) 36
V. Les fichiers structurés 38
Annexe 41

Chapitre 1 : Outils et structure générale d'un programme pascal

I. Outils

1. Editeur de code source

Un éditeur de code source est un programme d'édition de texte spécialement conçu pour l'édition du code source d'un programme informatique. Il peut s'agir d'une application autonome ou intégrée à un Environnement de Développement Intégré (**EDI** ou **IDE**) ou à un navigateur Web.

Un éditeur de code source comporte des fonctionnalités spécialement conçues pour simplifier et accélérer la saisie du code source, telles que : la coloration syntaxique, l'indentation, la saisie semi-automatique, la correspondance d'accolades, etc.

Dans le cadre de ce cours, nous utiliserons le logiciel **Pascal XE/MyPascal** comme éditeur de code.

2. Compilateur

Un compilateur est un logiciel qui lit et traduit un code source en code machine directement exécutable par le processeur de l'ordinateur. Généralement, le code source est écrit dans un langage de programmation (Pascal dans notre cas). Dans le cadre de ce cours, nous utiliserons le compilateur Free Pascal.

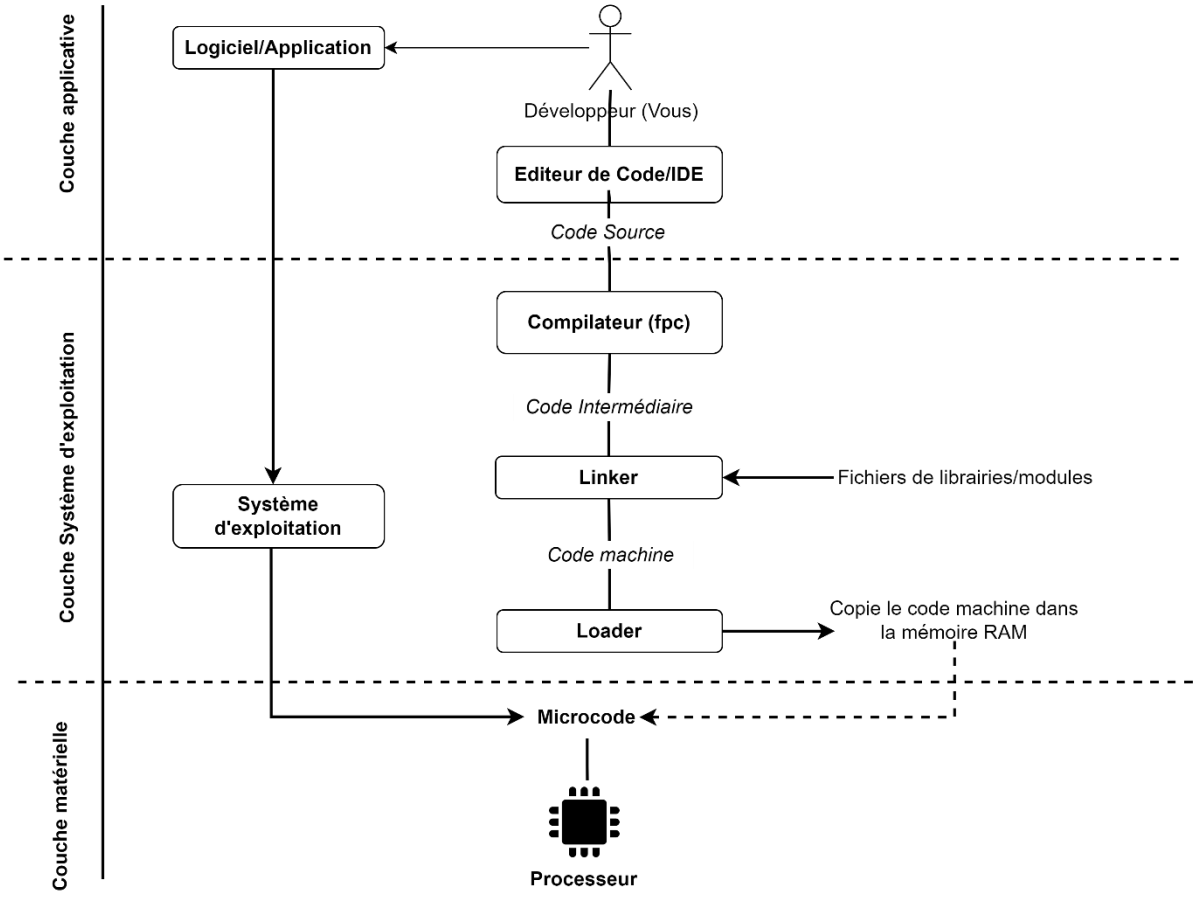


Figure 1 : Schéma simplifié de la compilation

II. Structure générale d'un programme en Pascal

1. Syntaxe d'un programme en pascal

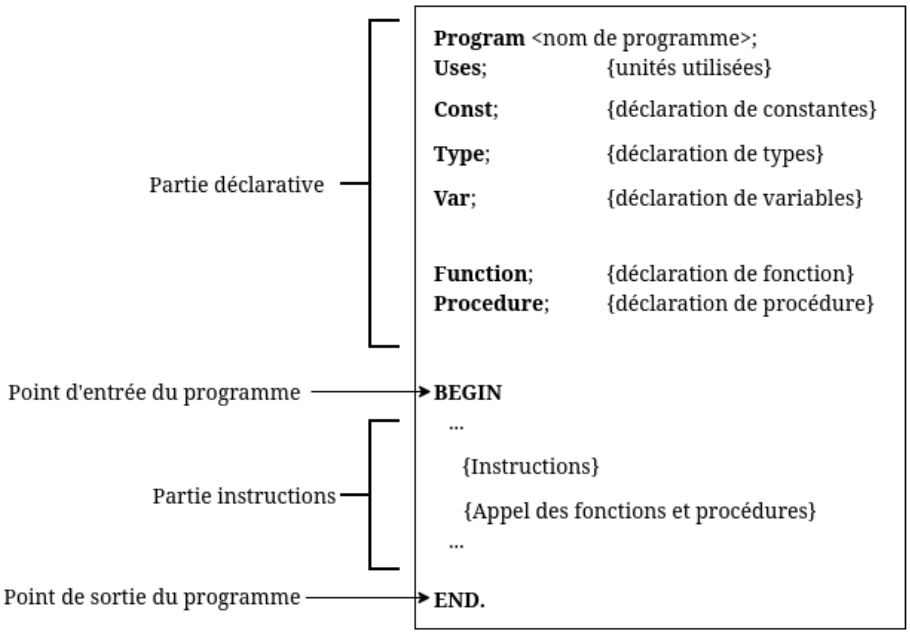


Figure 2 : Syntaxe générale d'un programme

Remarques :

- Le nom du programme doit respecter les règles liées aux identificateurs.
- Le compilateur Pascal ne différencie pas les majuscules des minuscules. Ainsi il est indifférent d'écrire **`begin`** ou **`Begin`**, voire **`begIn`** ou **`begiN`**. On dit qu'il n'est pas sensible à la casse.
- Chaque instruction doit se terminer avec un point-virgule. Il n'y a pas d'exception à la règle hormis **`Begin`** et l'instruction précédente **`End`** ou **`Else`**.
- Il est toléré de mettre plusieurs instructions les unes à la suite des autres sur une même ligne du fichier, mais il est recommandé de n'en écrire qu'une par ligne.

2. Premier programme

Un programme est une suite d'instructions, certaines étant des mots clés. Ce programme affiche la chaine de caractères « **`Bonjour, le monde`** » à l'écran :

```
Program Bonjour;
Uses Crt;
Begin
  Writeln('Bonjour, le monde');
End.
```

Code 1: Affichage de message à l'écran

3. Commentaires dans un programme

Nous pouvons insérer un commentaire dans un programme Pascal en entourant les lignes que l'on souhaite commenter par **{ }** ou en les précédant par **//** ou bien par **(* *)**.
Les commentaires servent à rendre le programme plus clair à la lecture, à noter des remarques, etc. et ne sont pas pris en compte à la compilation :

```
{Programme qui affiche bonjour}
PROGRAM Bonjour;
   Uses Crt;
BEGIN
   // Affiche Bonjour à l'écran
   Writeln('Bonjour');
END.
```

En informatique, une variable est une case mémoire, réservée pour stocker la future valeur d'une variable. Si cette valeur peut être modifiée au cours du programme, nous parlons de **variable**, dans le cas contraire, nous parlons de **constante**.

I. Les Types de variable

Il existe certains types de données qui sont définis par le compilateur Pascal comme des types de données standards. Ces types sont généralement les structures de données atomiques utilisées par un programme pascal.

Les différents types de variables sont :

- Integer (entier)
- Real (reél)
- Boolean (booléen)
- Char (caractère)

Tableau 1 : Résumé de quelques types utilisés en pascal

Type	Minimum	Maximum	Format
Integer	-32768	32767	Signé sur 16-bit
Shortint	-128	127	Signé sur 8-bit
Longint	-2147483648	2147483647	Signé sur 32-bit
Int64	-2^63	2^63 - 1	Signé sur 64-bit
Real	-	-	Signé sur 64-bit
Char ou Byte	0	255	Non signé sur 8-bit
Word	0	65535	Non signé sur 16-bit
String	0	4294967295	Non signé sur 32-bit

Un tableau détaillé des types utilisés en Pascal se trouve en annexe.

NB :

- Le langage Pascal ne différencie pas les majuscules et les minuscules. Ainsi **integer** est équivalent à **Integer** et à **INTEGER** : On dit qu'il n'est pas sensible à la casse.
- La liste exhaustive de tous les types disponibles en pascal peut être trouvée à l'adresse : https://wiki.freepascal.org/Data_type

II. Déclaration de variable et de constante

1. Choix des identificateurs

Les noms de constantes, variables, procédures, fonctions, tableaux, etc. (identificateurs) doivent suivre les mêmes conventions syntaxiques étudiées pendant le cours d'initiation à l'algorithmique et aussi, ils doivent impérativement être diffèrent de ceux des unités utilisées, des mots réservés du langage Pascal et ne doivent pas excéder une taille de 255 signes (1 lettres au minimum).

Exception :

Le caractère Underscore (_) est considéré comme une lettre. Il est donc autorisé à apparaitre au début, au milieu, à la fin, c'est-à-dire à n'importe quelle position dans un identificateur.
En pascal, les majuscules et les minuscules sont autorisés et sont équivalentes : On dit que le Pascal n'est pas sensible à la casse. Ainsi, en Pascal, Var et VaR désignent les mêmes objets.

Quelques mots réservés du langage pascal: `Program, Uses, Var, Begin, End, Write, Writeln, False, Case, True, If, Else, Function, Procedure, Or, Not, In, To, DownTo,` etc.

La liste complète est disponible au lien suivant : https://wiki.freepascal.org/Reserved_words.

2. Déclaration de variable

Une variable doit avoir un **nom** (ou identificateur), ainsi qu'un **type** (entier, réel etc...), qui définit la taille de la case mémoire. Ces deux attributs doivent être déclarés simultanément.

En Pascal, les variables doivent être déclarées avant le bloc d'instruction grâce au mot clé "**VAR**".

Syntaxe de déclaration de variable :

```
VAR identifiant_variable: type_variable;
```

Exemple : Avec une variable de nom a et de type entier :

```
 1. Program var_entier;
 2. Var
 3.   a : Integer; {Déclaration}
 4. Begin
 5.   {Affectation}
 6.   a := 5;
 7.
 8.   {Affichage de a}
 9.   Writeln('valeur de a = ', a);
10. End.
```

Remarque :

- Le choix de l'identificateur doit suivre les conventions syntaxiques étudiées pendant le cours d'initiation à l'algorithmique.

3. Déclaration d'une constante

On peut également utiliser des constantes, qui sont des valeurs qui restent fixes tout au long du programme et pour chaque exécution. Par convention, nous écrirons le nom des constantes en lettres capitales.

Syntaxe :

```
1. Const
2.     PI = 3.14;
3.     NOM = 'Koffi';
```

Le type de la constante est déterminé automatiquement par le compilateur (entier si nombre sans point, réel si nombre avec point, caractères si entre cotes).

Remarques :

- Lors de la déclaration, il est obligatoire de préciser la valeur qui sera associée à la constante durant tout le programme.
- On peut toujours utiliser un entier à la place d'un réel, mais pas l'inverse.
- Si on utilise un point décimal, il faut l'encadrer de chiffres (0.3 et non .3, 10 ou 10.0 et non 10.).

I. Les sorties

1. L'instruction Write

L'instruction **Write** permet d'afficher du texte et de laisser le curseur à la fin du texte affiché. Cette instruction permet aussi d'afficher les valeurs des variables, de constantes, de types ... Le texte à afficher doit être entre apostrophes. Si le texte à afficher contient une apostrophe, il faut alors la doubler. Les différents noms de variables doivent être séparés par des virgules.

Syntaxe :

```
1. Write('Une phrase d''exemple');
2. Write('qui affiche 2 + 2 = ', 2 + 2);
```

2. L'instruction WriteLn

L'instruction **WriteLn** est semblable à la précédente (**Write**) à la différence près que le curseur est maintenant renvoyé à la ligne suivante après écriture : On parle de retour chariot (Carriage Return).

Syntaxe :

```
1. WriteLn('Une ligne');
2. WriteLn('Une autre ligne');
```

II. Les entrées

1. L'instruction Read

L'instruction **Read** permet à l'utilisateur de rentrer une valeur qui sera utilisé par le programme. Cette instruction ne provoque pas de retour chariot, c'est-à-dire que le curseur ne passe pas à la ligne suivante.

Syntaxe :

```
Read(<variable1>, <variable2>);
```

Exercice 1 : Ecrire un programme qui récupère le nom, l'année de naissance et affiche le message «**Bonjour __, tu es né en __. Tu dois actuellement avoir __ ans** ! » à l'écran.

NB : L'utilisateur doit saisir le nom et l'année de naissance en une entrée.

2. L'instruction ReadLn

L'instruction **ReadLn** permet à l'utilisateur de rentrer une valeur qui sera utilisé par le programme. Cette instruction provoque un retour chariot, c'est-à-dire que le curseur passe à la ligne suivante. Lorsqu'aucune variable n'est affectée à l'instruction, il suffit d'appuyer sur la touche «**Entrée**».

Exercice 2 : Réécrire le programme de l'**exercice 1** en utilisant cette fois-ci l'instruction **ReadLn**. Cette fois-ci l'utilisateur saisit d'abord le nom et ensuite son année de naissance sera demandée.

III. Affectation d'une variable

Dans le corps du programme, une variable doit être impérativement **initialisée** avant toute utilisation. Deux moyens sont possibles :

1. Par lecture

Les instructions **Read** et **ReadLn** peuvent être utilisées pour lire et affecter une valeur à une variable. Les valeurs à lire des valeurs sont fournies à travers des saisies au clavier.

Exemple :

```
1. Writeln('Entrer les valeurs de a et b: ');
2. Readln(a, b);
```

Quand le programme s'exécute, on voit apparaître à l'écran : **'écrire les valeurs de a et b'.** Les deux valeurs alors écrites par l'utilisateur (séparées par un espace ou un retour à la ligne) sont affectées aux variables a et b à l'aide de l'instruction `readln(a,b);`.

2. Par affectation

Il est possible d'initialiser une variable en lui affectant une valeur directe ou une valeur provenant d'une expression. Cette affectation est réalisée en utilisant l'opérateur `:=`.

Dans le cas d'une expression, l'expression doit produire une valeur d'un type compatible ou du même type que la variable à affecter.

Exemple : `x := 1;  {signifie x reçoit la valeur 1}.`

Si les variables a et b ont été initialisées comme dans l'exemple précédent (au **3.1**), l'instruction `x := (a + b) / 2` affecte à x la valeur du calcul `(a + b) / 2`.

Attention :

- Ne pas confondre `:=` avec le `=` réservé aux données constantes et au symbole de comparaison.

Un opérateur est un symbole qui indique à l'ordinateur d'effectuer un certain nombre d'opérations mathématiques ou logiques. Pascal dispose des opérateurs suivants :

- Les opérateurs arithmétiques
- Les opérateurs relationnels
- Les opérateurs logiques
- Les opérateurs de bit à bit
- Les opérateurs d'ensembles
- Les opérateurs de chaines de caractères

La liste exhaustive peut être retrouvé ici : https://wiki.freepascal.org/Operators

I. Opérateurs et fonctions arithmétiques

Ces opérations dépendent du type des objets (variables ou constantes).

Tableau 2 : Quelques opérateurs arithmétiques

Opérateurs	Entrée(s)	Sortie	Commentaires
`+, -, *`	réel/entier	réel /entier	Addition, Soustraction, Multiplication
`/`	réel /entier	réel	Le type de sortie peut donc être diffèrent du type de l'entrée
`div`	entier	entier	Effectue la division entière d'un numérateur par un dénominateur : `37 div 5` donne `7`.
`mod`	entier	entier	Retourne le reste de la division de deux nombres.
`sin, cos, exp, ln, sqrt`	réel/entier	réel	`sqrt` est la racine carrée `sin` est le sinus `cos` est le cosinus `exp` l'exponentiel `ln` le logarithme
`sqr`	réel/entier	réel/entier	Retourne le carrée d'un nombre.
`trunc`	réel	entier	Retourne la partie entière d'un nombre réel.
`abs`	réel/entier	réel/entier	Retourne la valeur absolue.

II. Opérateurs relationnels

Le tableau suivant montre les opérateurs relationnels supportés par pascal. On suppose que la variable `A` possède la valeur `10` et la valeur de `B` est `20` :

Opérateur	Description	Exemple
=	Vérifie si la valeur des deux opérandes est égale ou non. Si les valeurs sont égales, la condition devient **true**, dans le cas contraire **false**.	(A = B) donne **true**
<>	Vérifie si la valeur des deux opérandes est égale ou non. Si les valeurs sont différentes, la condition devient **true**, le cas contraire, elle donne **false**.	(A <> B) donne **true**
>	Vérifie si la valeur qui se trouve à gauche de l'opérateur est plus grande que celle à droite. Si Oui, la condition devient **true**, le cas contraire, elle donne **false**.	(A > B) donne **false**
<	Vérifie si la valeur qui se trouve à gauche de l'opérateur est plus petite que celle située à droite. Si Oui, la condition devient **true**, le cas contraire, elle donne **false**.	(A < B) donne **false**
>=	Vérifie si la valeur qui se trouve à gauche de l'opérateur est plus grande ou égale à celle située à droite. Si Oui, la condition devient **true**, le cas contraire donne **false**.	(A >= B) donne **false**
<=	Vérifie si la valeur qui se trouve à gauche de l'opérateur est plus petite ou égale à celle de droite. Si Oui, la condition devient **true**, le cas contraire donne **false**.	(A <= B) donne **true**

Les variables d'entrées sont de type entier/réel ; la variable de sortie est de type booléen. Ces opérations sont principalement utilisées dans les instructions conditionnelles (que l'on reverra ultérieurement).

III. Opérateurs logiques

Opérateur	Opération
not	Opérateur de négation. Il est dit unaire.
and	ET logique
or	OU logique
xor	XOR logique

Ces opérateurs peuvent être utilisés dans les instructions conditionnelles.

Exemple : `(x < -1) AND (x > 1)` ou encore `(x > 0) OR (y = 0)`, peuvent être une condition du `if`.

IV. Opérateurs de bit à bit

Ce sont les opérateurs **not**, **and**, **or**, **xor**, **shl**, **shr**, **<<** (décalage à gauche), **>>** (décalage à droite).

V. Opérateur sur les chaines de caractères

Il est possible de combiner deux ou plusieurs chaines de caractères en utilisant l'opérateur de concaténation note **+.**

Exemple :

```
Writeln('Demain' + ' il ' + 'fera ' + 'beau');
```

Quelques fonctions applicables aux chaînes de caractères :

- **Concat**(s1, s2, s3, …, sn) : Concatène les chaînes de caractères **s1, s2**, etc. en une seule chaine. Cette fonction est semblable à l'opérateur **+**.
- **Copy(s, i, j)** : Extrait de la chaîne de caractères **s**, un nombre **j** de caractères à partir de la position **i**
- **Delete(s, i, j)** : Supprime dans la chaine **s**, un nombre **j** de caractères à partir de la position **i**.
- **Insert(s1, s2, i):** Insert la chaine **s1** dans la chaine **s2** a la position **i**.
- **Pos(s1, s2):** Renvoit la position de la sous-chaîne s1 dans la chaine mère **s2**. Si la sous-chaîne n'existe pas elle retourne **0**.
- **Str(x, s)**: Convertit le nombre (**Integer** ou **Real**) x en chaînes de caractères de nom **s.**
- **Val(x, s, error)**: Convertit la chaîne de caractères de nom **s** en un nombre (**Integer** ou **Real**) **x** et retourne un code erreur **error** de type **integer** qui est égale à **0** si la conversion est possible.
- **FillChar(s, n, i):** Introduit **n** fois dans la chaiine **s** la valeur **i** (de type **Byte** ou **Char**).

VI. Quelques opérations avec les caractères

Le jeu des caractères, comportant les lettres, les digits, l'espace, les ponctuations, etc., est codé sur un octet non signé. Le choix et l'ordre des 256 caractères possibles dépendent de la machine et de la langue. Sur PC, on utilise le code ASCII, où **'A'** est codé par 65, **'B'** par, 66, **'a'** par 97, **' '** par 32, **'{'** par 123, etc.

Les opérateurs sur les chars sont :

- **ord('c')** donne le numéro d'ordre dans le codage ascii du caractère **'c'** ;
- **chr(97)** donne le caractère dont le code ascii est **97**.
- **succ('c')** retourne le caractère suivant c dans l'ordre ascii, **chr**(**ord**('c') **+ 1**) .
- **pred('c')** retourne le caractère précèdent **'c'** dans l'ordre ascii.

Remarque : Il y a une erreur à l'exécution si le caractère n'existe pas.

Bonne et heureuse année 2025 à toi. Une année, c'est 365 jours, 8 760 heures, 525 600 minutes et 31 millions 536 000 secondes : Une énorme chance de faire les choses autrement. Je te souhaite un bon courage et la réussite.

Pascal dispose de type pré-déclarés dont les **booléens**, les **entiers** (**Integer**), les réels (**Real**) et les caractères (**Char**), etc. Il est aussi possible au programmeur de déclarer des types personnalisés.

I. Type Simple

Les types simples sont déclarés dans la partie déclarative du programme et avant la déclaration des variables utilisant ce nouveau type.

Syntaxe :

```
Type nom_du_type = nouveau_type;
```

Exemples :

```
 1. Program type_simple;
 2. Uses Wincrt;
 3. Type
 4.   chaine_20 = String[20];
 5.   chaine_50 = String[50];
 6. Var
 7.   nom: chaine_20;
 8.   prenom: chaine_20;
 9. Begin
10.   Write('Quel est ton nom? ');
11.   Readln(nom);
12.   Write('Quel est ton prenom? ');
13.   Readln(prenom);
14.
15.   Write('Bienvenu ', nom, ' ', prenom, '!');
16. End.
```

II. Type structuré

Nous pouvons être amenés à utiliser des types structurés. Ils nous permettent de ranger dans une seule variable des sous-variables nommées **champs**.

Syntaxe :

```
1. Type mon_du_type = Record
2.      sous_type1 : nouveau_type1;
3.      sous_type2 : nouveau_type2;
4.      sous_type3 : nouveau_type3;
5. End;
```

Exemple :

```
1. Type personne = Record
2.      nom : string[30];
3.      prenom : string[50];
4.      age : Integer;
5. End;
```

Remarque :

- Les champs sont placés dans un bloc **Record..End ;** et un sous-type peut lui-même être de type **Record**.

```
1. Type mon_du_type = Record
2.      sous_type1 : nouveau_type1;
3.      sous_type2 = Record
4.              sous_type2_1 : nouveau_type2;
5.              sous_type2_2 : nouveau_type3;
6.              sous_type2_3 : nouveau_type4;
7.      End;
8. End;
```

- L'accès aux différents champs peut se faire de manière indexée à l'aide d'un point **'.'**, ou à l'aide de la structure **WITH DO**.

```
1. personne.nom := 'ABALO';
2. personne.prenom := 'MA';
3. personne.age := 20;
```

Ou

```
1. With personne Do
2. Begin
3.     nom := 'ABALO';
4.     prenom := 'MA';
5.     age := 20;
6. End;
```

- La seule opération globale sur un enregistrement est : recopier le contenu d'un enregistrement **r2** dans un enregistrement **r1** en écrivant : **r2 := r1** (avec **r1** et **r2** de même type); Ceci est équivalent (et plus efficace) que de copier champ à champ ; en plus on ne risque pas d'oublier un champ.

Exercice 3 : Créer un type structure pouvant contenir les informations d'un étudiant (**nom**, **prenom**, **année de naissance** – jj/mm/aa, **sexe**, **age**) de l'IAI-TOGO, ensuite faire saisir les informations par un utilisateur.

III. Type intervalle

C'est un sous-ensemble de valeurs consécutives d'un type hôte. Ils peuvent être de type nombre entier (**Byte**, **Integer**, **ShortInt**, **LongInt**, **Long**, **Word**) ou caractères (**Char**). Un type Intervalle est forcément de type entier ou est compatible avec un type entier. Certaines fonctions sont réservées aux **types** intervalle, comme renvoyer le successeur dans l'intervalle considéré, sachant qu'un intervalle est forcément ordonné et continu.

Syntaxe :

```
Type mon_type = N..M;
```

N…M ou **N** et **M** sont des constantes du même type, et sont les bornes inférieures et supérieures de l'intervalle, **N** et **M** inclus.

Exemple :

```
1. Var
2.   pourcentage : 0..100;    {le type hôte est integer}
3.   alpha : 'A'..'Z';        {le type hôte est char}
4.   digit : '0'..'9';        {le type hôte est char}
5.   reponse : false..true;   {le type hôte est booléen}
```

Les instructions **Inc**(), **Dec**(), **Succ**(), **Pred**(), **Ord**() s'appliquent aux types intervalles qu'ils soient de type nombre, entier ou caractères et énumérés.

IV. Type énuméré

Un type énuméré est un type dont les variables associées n'auront qu'un nombre très limité de valeur (au maximum 256 différentes possibles). La définition d'un type énuméré consiste à déclarer une liste de valeurs possibles (256 au maximum) associées à un type. Une variable de ce type énuméré aura une et uniquement une seule de ces valeurs.

Il est fréquent en programmation que l'on ait à distinguer plusieurs cas, et que l'on cherche à coder le cas à l'aide d'une variable.

Exemple :

```
Var
        feux : 0..3; { rouge, orange, vert, clignotant }
Begin
        { ... }
        If feux = 0 Then
                Arreter
        Else If feux = 1 Then
                Ralentir
        Else If feux = 2
        { ... }
 End.
```

Ceci est très pratique, mais dans un programme un peu long cela devient rapidement difficile à comprendre, car il faut se souvenir de la signification du code. D'où l'intérêt d'utiliser un type énuméré, qui permet de donner un nom aux valeurs de code :

```
 1. Var
 2.  feux : (Rouge, Orange, Vert, Clignotant);
 3. Begin
 4.  { ... }
 5.  if feux = Rouge
 6.     then Arreter
 7.  else if feux = Orange
 8.     then Ralentir
 9.  else if feux = Vert
10.  { ... }
11. End.
```

1.1 Déclarer un type énuméré

Les noms de types sont déclarés entre les mots clés **Type** et **Var**.

Syntaxe : `nom_du_type = type;`

Exemple :

```
1. Type
2.    couleurs_feux_t = (Rouge, Orange, Vert, Clignotant);
3. Var
4.    feux : couleurs_feux_t;
```

De la sorte, couleurs_feux_t est un nom de type au même titre que **Integer** ou **Char**.

Exemples complets :

Exemple 1 :

```
 1. Program jours;
 2. Uses crt;
 3. Type
 4.   TJours = (lun, mar, mer, jeu, ven, sam, dim);
 5. Var
 6.   audj: TJours;
 7. Begin
 8.   audj := mar;
 9.   audj := Succ(mer);
10.   Inc(audj, 3);
11.   Case audj Of
12.     lun: Writeln('Le jour actuel est Lundi');
13.     mar: Writeln('Le jour actuel est Mardi');
14.     mer: Writeln('Le jour actuel est Mercredi');
15.     jeu: Writeln('Le jour actuel est Jeudi');
16.     ven: Writeln('Le jour actuel est Vendredi');
17.     sam: Writeln('Le jour actuel est Samedi');
18.     dim: Writeln('Le jour actuel est Dimanche');
19.   End;
20. End.
```

Exemple 2 :

```
1. Program portrait;
2. Const
3.     TailleMin = 50; { en cm }
4.     TailleMax = 250;
5. Type
6.     taille_t = TailleMin..TailleMax;
7.     couleurs_t = (Blond, Brun, Roux, Bleu, Marron, Noir, Vert);
8.     cheveux_t = Blond..Roux;
9.     yeux_t = Bleu..Vert;
10. Var
11.     taille_bob, taille_luc : taille_t;
12.     cheveux_bob, cheveux_luc : cheveux_t;
13.     yeux_bob, yeux_luc : yeux_t;
14. Begin
15.     taille_bob := 180;
16.     cheveux_bob := Brun;
17.     yeux_bob := Noir;
18. End.
```

Exercice 3 : Compléter l'exercice 2 en ajoutant les champs niveau, filière et classe de type énuméré.

I. Instructions

Une instruction est dite simple si elle se réduit à une seule commande : **Writeln**('bonjour') ou **s:=0;** Sinon, l'instruction est dite composée : on peut aussi parler de groupe d'instructions.

1. If … Then … Else

Cette instruction est l'équivalent de : **SI…ALORS**… **SINION**, et répond à l'attente : Si une condition est vraie (par exemple x ≠ 0), alors on veut effectuer une certaine instruction (par exemple, diviser par x) et sinon, on en effectue une autre.

Syntaxe:

```
Instructions A;
If condition  Then
Begin
  Instructions B;
End
Else
Begin
  Instructions C;
End;
Instructions D;
```

Remarque :

- Si les instructions B ou C sont simples (en une ligne), le **BEGIN END**; correspondant est facultatif.
- Le **ELSE** est facultatif ; mais il ne doit pas être précédé immédiatement d'un ';'

Attention : Ne pas confondre **x := 3** et **x = 3** lorsque x est une variable.

- **x := 3** est une affectation : elle correspond à l'instruction "donner à x la valeur 3".
- **x = 3** est une condition booléenne (proposition qui est vraie ou fausse) : elle correspond à la question "**x est-il égal à 3** ?".

Exemple :

```
Program IfElse;
Uses crt;
Var
  nombre: Integer;
Begin
  write('Entrez un nombre : ');
  Readln(nombre);

  If nombre < 100
    Then WriteLn(nombre, ' est inférieur à 100.') // Pas de ; ici
    Else WriteLn(nombre, ' est supérieur à 100.');
End.
```

Exercice 4: Modifiez le programme précédent en entourant les instructions du avec des **Begin** et des **End.**
Il est également possible d'insérer d'autres blocs **IF** dans un **ELSE** : On parle d'imbrication de si.
Exemple :

```
1. Program IfElseIf;
2. Uses crt;
3. Var
4.   nombre: Integer;
5. Begin
6.  Write('Entrez un nombre : ');
7.  Readln(nombre);
8.
9.  If (nombre < 50) Then
10.  Writeln(nombre, ' est inférieur à 50.')
11.  Else
12.   Begin
13.    If (nombre < 73) Then
14.     Writeln(nombre, ' est inférieur à 73 et supérieur à 50.')
15.    Else
16.     Writeln(nombre, ' est supérieur ou égale à 73.')
17.   End
18. End.
```

Les conditions booléennes sont généralement présentes dans les instructions conditionnelles (**if** then **else**) ou les boucles conditionnelles (**while** et **repeat until**).

Exercice 5 : Implémenter les algorithmes des exercices **6** et **8** (Page 8) du cahier d'exercices.

2. La structure Case…Of…End

Cette instruction compare la valeur d'une variable de type entier ou caractères à un ensemble de valeurs spécifiées par la structure **Case Of**.

Syntaxe :

```
1. CASE E OF
2.   C1 : Ia;
3.   C2 : Ib;
4.   C3, C4 : Ic; {liste}
5.   C5..C6 : Id; {intervalle}
6.   C7, C8, C9..10;
7.   Else Iautre; {en option}
8. End;
```

Cette instruction signifiant *choix selon* permet d'exécuter l'une des instructions **Ix** selon le cas **E**. **E** est une expression ordinale (dont le type est un entier, un caractère, un booléen, ou un énuméré, mais pas un réel ni une chaine de caractères). **Les Cx sont des constantes ordinales du même type que E**.

Exercice 5: Ecrire un programme qui lit un caractère, puis classe ce caractère comme espace, lettre, entier ou autre.

Exercice 6: Implémenter l'algorithme de l'exercice **9** (Page 8) du cahier d'exercices.

I. Boucle FOR

Cette instruction permet d'incrémenter une variable à partir d'une variable initiale jusqu'à une valeur finale et d'exécuter une ou plusieurs instructions après chaque incrémentation. Les valeurs doivent être des entiers (**Integer**) ou des caractères de la table ASCII (**Char**). Elle s'emploie pour répéter une suite d'instructions *n* fois, lorsque *n* est connu à l'avance.

Syntaxe :

```
1. instructions A;
2. FOR i := n1 TO n2 DO
3. Begin
4.   instructions B;
5. End;
6. instructions C;
```

Remarque :

- Le groupe d'instructions `B` est exécuté une première fois avec `i = n1`, ensuite une deuxième fois avec **`i = n1 + 1`**, puis une dernière avec **`i = n2`**. La variable compteur **`i`** doit être déclarée.
- `n1` et `n2` sont deux nombres entiers ou alors variables de type entier, déclarées et initialisées.
- Si l'instruction B est simple, le **Begin** et le **End**; correspondant n'est pas nécessaire.
- Si **`n1 > n2`**, le groupe d'instructions `B` n'est pas exécuté.

- ***Variante*** **:** Si l'on veut aller de *n2* à *n1* dans l'ordre décroissant, la syntaxe devient : **For** *i := n2* **DownTo** n1 **Do**.

Exercice 7 : Implémenter les algorithmes des exercices **10**, **11**, et **13** (Page 9) du cahier d'exercices.

II. Boucle conditionnelle

- Par opposition à l'instruction répétitive **For**, les instructions **While** et **Repeat** s'emploient dès que l'on ne connait pas à l'avance le nombre d'itérations à effectuer.

1. La boucle WHILE…Do

Elle répète une suite d'instructions tant qu'une certaine condition est vraie. Une variable de l'expression d'évaluation doit être sujette à modification au sein de la boucle. En l'absence de cette modification, la boucle ne se termine jamais : On parle d'une boucle infinie.

Syntaxe :

```
1. Instruction A;
2. While condition Do
3. Begin
4.    Instruction B;
5. End;
6. Instruction C;
```

Remarque :

- Les instructions B peuvent ne pas être exécutées du tout (si dès le départ la condition est fausse)
- Si l'instruction B est simple, le **Begin** et le **End**; correspondant est inutile.
- **Attention : Il faut s'assurer que la condition devient fausse au bout d'un certain temps. Sinon, le programme ne s'arrêtera jamais (boucle infinie).**

Exercice 8 : Écrire un programme qui affiche la table de multiplication complète.
Exercice 9 : Implémenter l'algorithme de l'exercice **17** (Page 10) du cahier d'exercices.

2. La boucle REPEAT...UNTIL

Cette boucle exécute premièrement les instructions placées entre deux bornes (**Repeat** et **Until**) et évalue à chaque répétition une condition de type booléenne afin de décider de l'arrêt ou de la continuité de la répétition. **Il y a donc au moins une fois exécution des instructions**.
Il est nécessaire qu'au moins une variable intervenant lors de l'évaluation de fin de boucle soit **sujette à modification à l'intérieur de la boucle**. En l'absence de modification, la boucle ne se termine jamais : On parle de boucle infinie.

Syntaxe :

```
1. Repeat
2.   instructions A;
3. Until condition;
```

- **Repeat Until** permet de délimiter l'instruction A, donc qu'elle soit simple ou composée, il n'y a pas besoin de **Begin End**;
- **Attention : Il faut s'assurer que la condition devient fausse au bout d'un certain temps. Sinon, le programme ne s'arrêtera jamais (boucle infinie).**

Exercice 10 : Ecrire un programme qui compte de **1** à **1000**.
Exercice 11 : Implémenter les algorithmes des exercices **12**, **14** et **15** (Page 9) du cahier d'exercices.

3. Arrêt de boucle et de programme

Il est possible de terminer les boucles **For**, **While** ou **Repeat** en cours d'exécution grâce à l'instruction **Break** lorsque celle-ci est placée au sein de la boucle en question. Et pour quitter le programme principal (Structure **Begin**…**End.**) ou un bloc sous-programme (Structure **Begin**…**End;**) l'instruction **Exit** est utilisé.

Exemple 1 : Ce programme affiche des nombres entiers de 0 à 4. Notons l'arrêt prématuré de la boucle avec l'instruction **Break**.

```
 1. Program arret_boucle;
 2. Uses crt;
 3. Var
 4.   i: Integer;
 5. Begin
 6.   For i := 0 to 10 Do
 7.   Begin
 8.    If i = 5 Then
 9.      Break;
10.    WriteLn(i);
11. End;
```

Exercice12 : Que fait le programme suivant ?

```
 1. Program arret_boucle;
 2. Var
 3.   i : Integer ;
 4. Begin
 5.   While(i <> 13) Do
 6.     Begin
 7.       Write('Entrez un nombre :
') ;
 8.       Readln(i) ;
 9.       Writeln(i) ;
10.       If i = 0 Then
11.         Exit;
12.     End;
13.   Writeln('Boucle terminée.') ;
14. End.
```

Un outil particulièrement pratique et proposé dans la grande majorité des langages est le tableau. Il permet le stockage d'une série d'éléments de même type.

L'intérêt du tableau est de pouvoir stocker en mémoire des données que l'on pourra retrouver grâce à d'autres valeurs à l'aide de boucles, de formules, d'algorithmes. On peut utiliser un tableau afin de représenter l'état d'un échiquier, le résultat d'une fonction mathématique... Il est possible d'introduire des variables de presque tous les types au sein d'un tableau : **Char**, **Integer**, **Real**, **String**, **Byte**, **Record**, etc.

I. Déclaration de tableaux

1. Tableaux à une dimension

Un tableau, tout comme une variable quelconque, doit être déclaré dans la partie déclarative du programme. On doit toujours spécifier le type des variables qui seront introduites dans le tableau.

Syntaxe :

```
VAR tableau : ARRAY[deb..fin] OF (type);
```

Note :

- Les valeurs **deb** et **fin** doivent être des **Integer** ou des **Char** (c'est-à-dire de type énuméré). **deb ≤ fin**. L'un ou les deux peuvent être négatifs. Le tableau comportera donc **fin-deb+1** cases, remplies par des éléments de type `type`.
- Chaque case du tableau est alors une **variable** à part entière, identifiée par `tableau[i]`. Au départ, le tableau est **vide** : il faudra donc penser à initialiser toutes les cases du tableau avant de l'utiliser.

Exemples :

```
1. Var tab1 : Array[0..10] Of Byte;
2. Var tab2 : Array[1..100] Of Integer;
3. Var tab3 : Array[-10..10] Of Real;
4. Var tab4 : Array[50..60] Of String;
5. Var tab5 : Array['A'..'S'] Of Char;
6. Var tab6 : Array['a'..'z'] Of Extended;
```

Imaginons un professeur voulant stocker les notes de 5 élèves, il peut ainsi utiliser un tableau d'entiers de longueur 5. Illustrons ce dernier :

```
Var
    notes : array[1..5] of Integer;
```

2. Tableau à plusieurs dimensions

Un tableau peut avoir plusieurs dimensions.

Syntaxe :

```
Var
  nom_du_tableau : Array[MinDim1..MaxDim1, MinDim2..MaxDim2] Of type;
  nom_du_tableau : Array[MinDim1..MaxDim1, MinDim2..MaxDim2, MinDim3..MaxDim3] Of type;
```

Remarque :
L'accès aux différentes valeurs du tableau se fait en précisant les index des différentes dimensions du tableau : Par exemple `nom_du_tableau[index1, index2] := valeur;` avec **index1**, **index2** du même type énuméré que celui utilisé pour la déclaration du tableau.

Exemple :

```
1. Var tab1 : Array[0..10, 0..10] Of Byte;
2. Var tab2 : Array[0..10, 0..100] Of Integer;
3. Var tab3 : Array[-10..10, -10..10] Of Real;
4. Var tab4 : Array[5..7, 20..22] Of String;
5. Var tab5 : Array[1..10, 1..10, 1..10, 0..2] Of Char;
```

Exercice 13 : Implémenter les différents algorithmes de recherche sur les tableaux vus au cours d'initiation à l'algorithmique.
Exercice 14 : Implémenter l'algorithme de l'exercice **2** (Pg. 13) du cahier d'exercices.

II. Créer un type "tableau"

Rappel : la syntaxe d'une fonction comme d'une procédure demande la connaissance du type du paramètre d'entrée. Par exemple, `Function f(x : Real): Integer;`

Pour pouvoir appliquer des fonctions ou des procédures à un tableau, il faut donc **déclarer le type** de ce tableau en amont. La déclaration d'un nouveau type se fait juste avant la déclaration des variables.

Syntaxe :

```
Type polynome = Array[0..10] Of real;
```

Ensuite, se fait la **déclaration des variables** : `Var P, Q, R : polynome ;`

Dès que l'on a défini un tel type polynôme, on peut appliquer une fonction ou une procédure à un polynôme. Attention de garder en mémoire qu'une fonction ne pourra jamais renvoyer un polynôme : sa sortie ne peut être qu'un réel (ou un entier). Mais une procédure pourra, par son action, changer la variable polynôme d'entrée.

III. Les tableaux dynamiques

Un tableau dynamique est un tableau dont les dimensions ne sont pas connues pendant le temps de la compilation (Compile-time).

1.1 Tableau à une dimension

Syntaxe :

```
Var notes = Array Of Char;
```

Une fois le tableau dynamique à une dimension déclarée, on pourra utiliser la procédure **SetLength** pour changer la taille du tableau à la taille désirée.

Exemple :

```
1. Program dynamic_table;
2. Uses crt;
3. Var
4.   notes : Array Of Integer;
5. Begin
6.   SetLength(notes, 300);
7.   WriteLn('Low(notes) = ', Low(notes), ', High(notes) = ', High(notes));
8. End.
```

1.2 Tableau à deux dimensions

Syntaxe:

```
Var notes = Array Of Array Of LongInt;
```

Exemple :

```
1. Program dynamic_table;
2. Uses crt;
3. Var
4.   notes : Array Of Array Of Integer;
5. Begin
6.   SetLength(notes, 12, 300);
7.   WriteLn('Low(notes) = ', Low(notes), ', High(notes) = ', High(notes));
8. End.
```

Remarque : Il est possible d'utiliser les procédures **Delete** et **Insert** pour supprimer et insérer des valeurs dans les tableaux dynamiques.

Plus de détail ici https://wiki.freepascal.org/Dynamic_array

IV. Quelques opérations sur les tableaux

La seule opération globale possible sur les tableaux est l'affectation. Si R et A sont deux tableaux de même dimension et type de contenu, la commande `R:=A;` remplace le contenu du tableau R par celui du tableau A.

Toutes les autres opérations portant sur les tableaux se font **case par case**, donc en utilisant des boucles. Ce défaut engendre souvent des temps de traitement importants, en particulier pour les tableaux de grande taille.

Rappel : Toutes les cases d'un tableau sont des variables donc doivent être initialisées par lecture ou affectation.

1. Tableaux à une dimension

Opérations	Syntaxes
Déclarer un tableau T de taille 50 pouvant contenir des réels	`Var T : Array[1..50] Of Real;`
Déclarer un type tableau puis une variable T de ce type	`Type tab = Array[1..50] Of Real;` `Var T: tab;`
Initialiser la 15e case de T à 0	`T[15] := 0;`
Initialiser la 18e case de T à une valeur entrée par l'utilisateur	`WriteLn('Entrer la valeur de la case 18');` `ReadLn(T[18]);`
Afficher la valeur de la 25e case du tableau T	`WriteLn(T[25]);`
Initialiser tout le tableau T à 0	`For k:=1 To 50 Do T[k] := 0;`

Initialiser tout le tableau T à des valeurs entrées par l'utilisateur	`For k:=1 To 50 Do`
	`Begin`
	`  WriteLn('Entrer la valeur de la case ', k);`
	`  ReadLn(T[k]);`
	`End;`

Afficher tout le tableau T	`For k:=1 To 50 Do WriteLn(T[k]);`

2. Tableaux à deux dimensions

Opérations	Syntaxes
Déclarer le tableau T	`Var U : Array[1..50, 1..40] Of Real;`
Déclarer le type puis une variable T de ce type	`Type tab = Array[1..50] Of Real;` `Var T: tab;`
Initialiser la case située à la 11e ligne et 23e colonne de U à 0	`U[11,23]:=0;`
Initialiser la case située à la 8-ieme ligne et 27e colonne de U à une valeur entrée par l'utilisateur	`WriteLn('Entrer la valeur de la case située ligne 8 colonne 27');` `ReadLn(U[8, 27]);`
Afficher la valeur de la case située à la ligne 25, colonne 17 du tableau	`Writeln(U[25, 17]);`

Initialiser tout le tableau U à 0	`For i:=1 To 50 Do`
	`  For j:=1 To 40 Do`
	`  U[i,j]:= 0;`

Initialiser tout le tableau U avec des valeurs entrées par l'utilisateur	`For i:=1 To 50 Do`
	`  For j:=1 TO 40 Do`
	`  Begin`
	`  WriteLn('Entrer la valeur ligne ',i,'colonne ',j);`
	`  ReadLn(U[i,j]);`
	`  End;`

Afficher tout le tableau	`For i:= 1 To 50 Do`
	`  For j:=1 To 40 Do`
	`  WriteLn(U[i,j]);`

I. Notions de sous programmes

Il peut arriver que l'on ait à utiliser une même séquence d'instructions à différents endroits d'un programme. Il est alors judicieux de créer un sous-programme (fonction ou procédure) dont le code sera défini une fois pour toutes dans l'étape 3, et que l'on appellera dans le corps du programme aux différents endroits souhaités.

Avantage : le corps du programme est plus court et plus lisible puisque l'on évite ainsi des répétitions de code.

Exemple : Le calcul de nk décrit par la formule $nk = \frac{n!}{k!(n-k)!}$, nécessite le calcul de 3 factorielles ; on pourra écrire une fonction factorielle en amont, et ensuite l'appeler 3 fois.

Un sous-programme a la même structure qu'un programme, mais le **END** du corps du sous-programme est suivi d'un '**;**' et non d'un '**.**'. Il faut déclarer ces sous-programmes, juste avant le corps du programme.

Toutes les variables utilisées dans un sous-programme doivent être définies soit comme des paramètres d'entrée, soit comme des variables locales (les variables du programme sont appelées variables globales). Mais l'appel du sous-programme se fait dans le corps (**Begin…End.)** du programme.

Nous avons deux catégories de sous-programmes : **les procédures** et les **fonctions**.

II. Les procédures

Une procédure est un sous-programme. Écrire des procédures permet de découper un programme en plusieurs morceaux. Chaque procédure définit une nouvelle instruction, que l'on peut appeler en tout endroit du programme. On peut ainsi réutiliser le code d'un sous-programme sans limites.

1. Procédure sans paramètre

1.1 Principe

Il s'agit simplement d'isoler et de donner un nom à un groupe d'instructions. Ensuite, l'appel de ce nom à travers divers endroits du programme provoque à chaque fois l'exécution de ce groupe d'instructions.

Syntaxe :

```
1. Procedure NomDeLaprocedure;
2. Begin
3.   {Corps de la procédure}
4. End;
```

Exemple:

```
 1. Program procedure_carree;
 2. Var
 3.   nombre: Integer;
 4.   resultat: Integer;
 5. Procedure carree;
 6. Begin
 7.   resultat := nombre * nombre ;
 8.   writeln('Le carrée de ', nombre, ' est ', resultat);
 9. End;
10. Begin
11.   write('Entrer un nombre: ');
12.   readln(nombre);
13.   carree(); //premier appel de la procédure carrée
14.
15.   write('Entrer un autre nombre: ');
16.   readln(nombre);
17.   carree(); //deuxième appel la procédure carrée
18. End.
```

Remarques :

- Le nom de la procédure est un identificateur
- On déclare la procédure avant le **Begin** du programme principal
- On peut appeler une procédure **P1** depuis une procédure **P2**, mais il faut que la procédure **P1** ait été déclarée avant la procédure **P2**.
- Une procédure peut s'autoappeler : **C'est la récursivité**

1.2 Variables locales

Les objets du programme qui ne sont utiles que dans la procédure peuvent être définis dans les déclarations locales de la procédure.

Exemple :

```
 1. Program procedure_carree;
 2. Var
 3.   nombre: Integer;
 4. Procedure carree;
 5. Var
 6.   resultat: Integer;
 7. Begin
 8.   resultat := nombre * nombre ;
 9.   writeln('Le carrée de ', nombre, ' est ', resultat);
10. End;
11. Begin
12.   // Appels et affichage
13. End.
```

Remarques :

- Une variable déclarée localement n'existe que pendant l'exécution de la procédure, et ne sert qu'à cette procédure.
- Le programme principal n'a jamais accès à une variable locale de procédure.
- Une procédure n'a jamais accès à une variable locale d'une autre procédure.

NB : Les variables déclarées dans le **VAR** du programme principal sont appelées variables globales. Elles existent pendant toute la durée du programme et sont accessibles de partout. Une variable locale à une procédure **P**, portant le même nom **x** qu'une variable globale, masque la variable globale durant l'exécution de **P**.

2. Procédure paramétrée

Une procédure peut attendre un certain nombre de paramètres. Ces paramètres doivent lui être fournis pendant les appels et doivent aussi respecter les types annoncés lors de la déclaration de cette procédure.

Il existe deux types de passage de paramètre à une procédure : **Le passage par valeur** et le **passage par adresse.**

2.1 Passage par valeur

Ce type de passage est utilisé lorsqu'on ne voudra pas que le paramètre d'entrée puisse être modifié par la procédure. À l'appel, le paramètre est une variable ou une expression et c'est la valeur qui est transmise. Elle sert à initialiser la variable correspondante dans la procédure.

```
Procedure NomDeLaprocedure(NomsParametres : TypeDuParametres);
```

Remarques :

- Le paramètre d'entrée pendant l'appel de la procédure peut être juste un nombre, un réel ou autre selon le type attendu (et non nécessairement une variable).
- Dans le cas de plusieurs paramètres, les séparés par un point-virgule.

2.2 Passage par adresse ou référence

Ce type de passage est utilisé lorsqu'on a l'intention de modifier le paramètre d'entrée à l'intérieur de la procédure. À l'appel, le paramètre est une variable uniquement (jamais une valeur ou une expression). L'adresse mémoire (La référence) de la variable est transmise, non sa valeur. La variable utilisée dans la procédure est en fait la variable de l'appel, mais sous un autre nom, un alias.

Syntaxe :

```
1. Procedure NomDeLaProcedure(VAR NomDuParametre:TypeDuParametre);
2. Var Déclaration des variables LOCALES (ou constantes via CONST);
3. Begin
4.  {Corps de la procédure}
5. End;
```

Remarque :

- Dans le cas du passage par adresse, le paramètre d'entrée pendant l'appel de la procédure doit obligatoirement être une variable ayant le même type que celui déclaré.

III. Les fonctions

Une fonction est une procédure qui renvoie un résultat, de manière à ce qu'on puisse l'appeler dans une expression. Elle fournit un résultat à partir des données qu'on lui apporte (les paramètres d'entrées) : la notion de fonction en Pascal est assez proche de la notion de fonction en mathématiques.

1. Fonctions sans paramètre

1.1 Principe

Syntaxe :

```
1. Function NomDeLaFonction():Typesortie;
2. Begin
3.   {Corps de la fonction (liste des instructions) }
4.   NomDeLaFonction := ValeurSortie;
5. End;
```

Remarques :

- **NomDeLaFonction** n'est pas une variable.
- Une fonction doit toujours avoir un résultat.
- Les types de passages étudiés précédemment s'appliquent également aux fonctions.

Exemple :

```
 1. Program Fonction;
 2. Function Pi2(): Real;
 3. Const
 4.   PI = 3.14159265359;
 5. Begin
 6.   Pi2 := 2 * PI;
 7. End;
 8. Begin
 9.  Writeln(Pi2:0:2)
10. End.
```

2. Fonctions avec paramètres

Syntaxe :

```
1. Function NomDeLaFonction(NomDuParametre:TypeDuParametre):Typesortie;
2. Var Déclaration des variables locales ou constantes via CONST;
3. Begin
4.  {Corps de la fonction (liste des instructions)}
5.  NomDeLaFonction := ValeurSortie;
6. End;
```

NB : Il peut y avoir plusieurs paramètres d'entrées dans une fonction. Ils seront alors séparés par un "**;**" dans la déclaration, mais par une "**,**" lors de l'appel de la fonction dans le corps du programme.

Exemple :

```
 1. Program fonction;
 2. Function calcul(nombre : Real): Real ;
 3. Begin
 4.     calcul := 3 * sqr(nombre) + 2; {Retour du résultat}
 5. End;
 6. Var
 7.     nombre : Real ;
 8. Begin
 9.     Wrtie('Donnez un nombre') ;
10. Readln(nombre) ;
11.
12. Writeln('le resultat est', calcul(3)); {Appel de la fonction calcul}
13. End.
```

Exercice 15 : Écrit un programme qui permet d'effectuer le calcul $\boldsymbol{nk} = \frac{\boldsymbol{n}!}{\boldsymbol{k}!(\boldsymbol{n}-\boldsymbol{k})!}$. On demandera à l'utilisateur de saisir **n** et **k**.

Exercice 16 : Réorganiser les programmes écrits a l'exercice 13 et aux exercices **2** (Pg. 11), **5** (Pg. 17) en procédures et fonctions.

IV. Procédures et fonctions prédéfinies

Pascal comme la majorité des langages de programmation vient par défaut avec un certain nombre de procédures et fonctions dites standards.

1. L'unité Crt

Cette unité contient les routines d'affichage d'écran de texte. Quelques procédures et fonctions définies par l'unité crt.

- **Clrscr**

La procédure Clrscr permet d'effacer l'écran.

```
1. Writeln('Appuyer sur entrer pour effacer l'ecran !');
2. Readln;
3. ClrScr();
```

- **TextColor**

Cette procédure permet de changer la couleur du texte affiché dans la console.

```
1. TextColor(red);
2. Writeln('Ce texte est écrit en rouge');
```

- **Textbackground**

Cette procédure permet de changer la couleur de fond d'écriture du texte de la console.

```
1. Textbackground(green);
2. Writeln('Ce texte sera écrit sur un fond vert');
```

La liste complète peut être retrouvée à l'adresse web https://wiki.freepascal.org/Crt

2. Génération de nombres aléatoires

Il est quelquefois nécessaire d'avoir recours à l'utilisation de nombre qui soient le fruit du hasard. Cependant, l'ordinateur n'est pas capable de créer du vrai hasard. Il peut cependant fournir des données dites aléatoires, c'est-à-dire issues de calculs qui utilisent un grand nombre de paramètres eux-mêmes issus de l'horloge interne. On appelle cela un pseudo-hasard, car il est très difficile de déceler de l'ordre et des cycles dans la génération de valeur aléatoires. Pascal offre la possibilité de générer des nombres aléatoires.

Pour utiliser le générateur de nombres pseudo-aléatoires de Pascal, il faut insérer, au début du corps du programme, la commande **`Randomize.`** Elle permet d'initialiser le générateur aléatoire tout comme il faut initialiser la carte graphique pour faire des dessins.

Syntaxe :

```
Randomize;
```

Ensuite, on peut générer :

- Un réel aléatoire compris entre `0` et `1` grâce à la fonction **`Random.`**
- Un entier aléatoire compris entre `0` et `n-1` grâce à la fonction **`Random(n)`**.

Pour les mordus (Attention : Si tu as peur du clavier, ne lit pas ce truc) :

Attention ! Ne parle pas de ce truc pendant le cours.

Ecrit un jeu (programme) qui peut se jouer entre un utilisateur et une machine. La machine devra générer un nombre aléatoire et le garder secret. L'utilisateur devra deviner ce nombre secret généré par l'ordinateur.

L'utilisateur a droit à plusieurs tentatives et les règles du jeu sont les suivantes :

- S'il trouve le nombre secret à la première tentative on affichera le message **'Bravo, tu as trouvé le nombre secret à la première tentative. Sorcier va !'**
- Si le nombre proposé par le joueur est correct on affichera le message '**Enfin, tu as trouvé le nombre secret en x tentatives. Il commençait à faire nuit !**'.
- Si le nombre proposé par le joueur est plus grand que le nombre secret, on affiche le message **'Plus petit ! Toi aussi va doucement.'**
- Si le nombre proposé par le joueur est plus petit que le nombre secret, on affiche le message **'Plus grand ! Fait un effort tu peux y arriver.'**

Pour ceux qui n'ont pas froid aux yeux : Rendre le jeu multi-joueurs comme suit :

- D'abord, calme-toi !
- Demande au départ le nombre de joueurs
- Demande le nom de chaque joueur (**Tu peux utiliser les types enregistrement ou Record. Dit merci !**)
- Fait jouer le jeu à chaque joueur (**Je te défis de faire 3 parties par joueur. Tu acceptes le défi ? Si oui démerde-toi ! Si non poltron !**)
- Pense à un système de scores et attribue des scores à chaque joueur. **(**Par exemple tu peux avoir un score au départ qui est de **100 points**. Chaque tentative coute **10** points au joueur**. Il perd la partie s'il épuise ses 10 tentatives)**
- Désigne le gagnant et affiche le classement final avec le score obtenu par chaque joueur **(Utilise les couleurs et fait un bel affichage. Je compte sur toi).**
- **Tu as fini de faire tout ça ou tu es en train de lire seulement ?**

Pour ceux qui ne veulent pas dormir la nuit :

Si tu n'es pas fatigué de lire cette page, voilà encore une idée : Tu peux implémenter ce jeu en **C** ou en **Python** en utilisant une bibliothèque graphique. **SDL2** pour **C** et **PyGame** pour Python. Ou tu veux te tuer avec **Flutter ?** Vas-y, tu iras dans l'enfer des programmeurs.

I. Déclaration

Pour utiliser un ou des fichiers tout au long d'un programme ou dans une procédure, il faudra l'identifier par une variable dont le type est fonction de l'utilisation que l'on veut faire du fichier.

Il existe trois types de fichiers : les fichiers textes, Les fichiers types ou binaires et Les fichiers simples.

II. Les fichiers textes (Text)

Les fichiers textes (**Text**) sont écrits au format texte (chaînes de caractères, nombres) dans lesquels l'on peut écrire et lire ligne par ligne ou à la file avec les procédures **Write**(**Ln**) et **Read**(**Ln**).

Chaque fin de ligne du fichier se termine par les caractères 10 et 13 de la table ASCII (qui signifient respectivement Retour ligne (Line Feed, **LF**) = #10 et Retour chariot (Carriage return, **CR**) = #13).

Syntaxe :

```
Var f : Text;
```

III. Les fichiers types (File of)

Les fichiers typés (**File** of) sont des fichiers dans lesquels les données sont écrites telles qu'elles se présentent en mémoire : On dit qu'elles sont écrites dans leur représentation binaire.

La taille du fichier résultera directement et exactement de la taille en mémoire qu'occupe telle ou telle variable. Cela accroît la vitesse d'accès aux données du fichier. Mais le plus grand avantage, c'est que l'on obtient ainsi des fichiers parfaitement formatés, c'est-à-dire qu'on peut y lire et écrire directement des variables de type structuré qui contiennent plusieurs champs de données sans avoir à se soucier des divers champs qu'elles contiennent.

Syntaxe :

```
Var f : File Of {type};
```

Exemple :

```
1. Var f : File Of Integer;
2. Var f : File Of Real;
```

IV. Les fichiers simples (File)

Les fichiers (**File**) sont des fichiers dont le contenu n'est pas déterminé. Ils peuvent servir à faire une simple copie d'un fichier dans un autre, pour stocker des données de différents types (en se donnant les moyens de retrouver ce que sont ces types lors de la lecture), pour sauver tout le contenu d'une zone mémoire pour analyse ultérieure, etc.

Syntaxe :

```
Var f : File;
```

1. Lecture et écriture

1.1 Assignation

Avant de travailler sur un fichier, il faut le déclarer en lui affectant une variable qui servira à désigner le fichier tout au long du programme ou de la procédure dans laquelle il est utilisé. **Assign** s'applique à tous les types de fichiers.

Syntaxe :

```
Assign(variable_d_appel, nom_du_fichier);
```

1.2 Création et ouverture d'un fichier texte

Après avoir déclaré une variable de type fichier et assigné cette variable à un nom de fichier, on peut l'ouvrir en écriture. Cela a pour effet de créer un nouveau fichier vide.

Attention :

- **L'utilisation de Rewrite avec un fichier existant ouvre ce fichier, mais le vide**
- **On ne peut pas lire dans un fichier Text ouvert avec Rewrite**

L'ouverture en écriture, se fait par la procédure **rewrite**, comme suit :

Syntaxe :

```
Rewrite(f);
```

Remarque : Pour ouvrir un fichier en extension il faut utiliser la procédure **Append.**

1.3 Ecriture d'une ligne

Dès que le fichier est ouvert en écriture, on peut y écrire du texte en utilisant la procédure **Writeln**. Voici, par exemple, comment créer un fichier texte 'exemple.txt' d'une ligne :

```
1. Program Ecriture;
2. Var f: Text;
3. Begin
4.   Assign(f,'exemple.txt');
5.   Rewrite(f);
6.   Writeln(f,' Ceci est un fichier d''une ligne.');
7.   Close(f);
8. End.
```

La procédure **Close**() appelée à la fin sert à fermer le fichier.

1.4 Lecture d'un fichier texte

- **Ouverture d'un fichier texte en lecture**

Pour ouvrir un fichier en lecture, on utilise la procédure `reset` de la manière suivante :

Syntaxe:

```
Reset(variable_d_appel);
```

Attention :

- **On ne peut pas écrire dans un fichier Text ouvert avec Reset**

Il est possible de rouvrir un fichier Text en cours de programme, même s'il a déjà été refermé, grâce à sa variable d'appel. La position courante du curseur sera alors positionnée à la fin du fichier. Cela ne fonctionne qu'en écriture et qu'avec un **Text**.

Syntaxe :

```
Append(variable_d_appel);
```

- **Lecture d'une ligne et test de fin de fichier**

Pour lire une ligne dans le fichier, on utilise la procédure **readln**, qui affecte la ligne courante à une variable L de type chaine de caractères :

Syntaxe :

```
Readln(variable_d_appel, L);
```

Il s'agit ici d'un accès séquentiel, donc une lecture de ligne se fait nécessairement juste après la ligne lue précédemment.

Juste après l'ouverture du fichier, **readln** transfère la première ligne du fichier dans la variable L. À la deuxième exécution de cette procédure, L contiendra la 2e ligne du fichier et de manière générale, après la n-ième exécution, L contiendra la n-ième ligne du fichier.

Remarque : La lecture au-delà de la ligne d'un fichier provoquera une erreur. Pour éviter cette erreur, il faut utiliser la fonction **`Eof (End Of File)`** qui permet de tester si l'on se trouve en fin de fichier.

Exemple :

```
 1. PROGRAM fichier;
 2. VAR
 3.    f : Text;
 4.    L : String;
 5. Begin
 6.    Assign(f,'exemple.txt');
 7.    Reset (f);
 8.
 9.    While Not Eof (f) Do
10.    Begin
11.       Readln (f, L);
12.    End;
13.    CloseFile (f);
14. End.
```

- **Résumé des opérations et procédures sur un fichier** :

Tableau 2: Tableau des correspondances entre procédures et types de fichiers

Syntaxe	Lecture	Ecriture
`Reset(f)`	Text File of File	File of File
`Rewrite(f)`	File of File	Text File of File

Tableau 3: Tableau des correspondances entre procédures et types de fichiers

Syntaxe	Types de fichiers associés
`WriteLn(f)`	**`Text`**
`ReadLn (f)`	**`Text`**
`Write(f)`	**`Text`** **`File of`**
`Read(f)`	**`Text`** **`File of`**

V. Les fichiers structurés

Un fichier structuré est un fichier constitué d'enregistrements de même type.

L'écriture d'un enregistrement se fait avec la procédure **write**. Plus précisément, **write**(f, v) écrit la valeur de la variable v dans le fichier f à la suite du dernier enregistrement lu ou écrit. Bien entendu, la variable doit être de même type que les enregistrements du fichier.

Si le fichier vient d'être ouvert, l'écriture se fait dans le premier enregistrement.

Déclarons par exemple deux structures de type Livre :

```
Var vb, gaston : Livre ;
```

Donnons des valeurs à leurs champs :

```
1. vb.Titre :='Visual Basic 6, Le guide du programmeur';
2. vb.Auteur := 'Frantz';
3. vb.AnneePub := 2000;
4. gaston.Titre := 'Gaston Lagaffe - Des Gaffes et des Dégats';
5. gaston.Auteur :='Franquin';
6. gaston.AnneePub := 1975;
```

Après les instructions suivantes :

```
1. AssignFile(f,'Bibliotheque.dat');
2. Rewrite(f);
3. Write(f, gaston);
4. Write(f, vb);
```

Le fichier "Bibliotheque.dat" contiendra deux enregistrements de type Livre. Plus précisément, le premier enregistrement contiendra le livre Gaston et le deuxième, le livre **vb**.

1. Lecture séquentielle

La lecture d'un enregistrement à la suite du dernier enregistrement lu ou écrit se fait par la procédure **Read**. Plus précisément, pour affecter la valeur d'un enregistrement à une variable v, on écrira **Read**(f, v). Cela suppose bien entendu que la variable est du même type que les enregistrements du fichier.

Si le fichier vient d'être ouvert, l'enregistrement lu sera le premier. Pour les récupérer, déclarons deux variables de type Livre :

```
Var livre1, livre2, livre3 : Livre;
```

Après les instructions suivantes :

```
1. Assign(f,'Bibliotheque.dat');
2. Reset(f);
3. Read(f, Livre1);
4. Read(f, Livre2);
```

2. Lecture jusqu'en fin de fichier

De manière générale, on ne connait pas le nombre d'enregistrements contenus dans un fichier structuré. On utilise alors la fonction **Eof** pour savoir si on est arrivé à la fin du fichier.

Voici comment lire séquentiellement un fichier structuré afin d'effectuer un traitement sur chacun de ses enregistrements :

```
 1. Program Bilio;
 2. VAR f: File Of Type;
 3.     x : Type;
 4. Begin
 5.   Assign(f,'Bibliotheque.dat');
 6.   Reset(f);
 7.   While Not Eof (f) Do
 8.   Begin
 9.     Read(f, x);
10.   End;
11. End.
```

Annexe

Désignation	Description	Valeur minimale	Valeur maximale	Format (Octet)
Integer	Nombre entier sans virgule	Dépendant de la plateforme	Dépendant de la plateforme	-
Shortint (int8)	Entier	-128 (-2^7)	127 ($2^7 - 1$)	**1**
Byte (uint8)	Entier	0 (0)	255 ($2^8 - 1$)	**1**
Smallint (int16)	Entier	-32768 (-2^{15})	32767 ($2^{15} - 1$)	**2**
Word (uint16)	Entier	0 (0)	65535 ($2^{16} - 1$)	**2**
Longint (int32)	Entier	-2147483648(-2^{31})	2147483647($2^{31} - 1$)	
Longword (Cardinal, dword)	Entier	0 (0)	4294967295($2^{32} - 1$)	**4**
Int64	Entier	-2^{63}	$2^{63} - 1$	**8**
Qword (uint64)	Entier	0 (0)	$2^{64} - 1$	**8**
Single	Réels	1.5E-45	3.4E+38	**4**
Real	Réels	2.9E-39	1.7E+308	**8**
Double	Réels	5.0E-324	1.7E+308	**8**
Boolean	Variable booléenne	TRUE ou False	255($2^8 - 1$)	**1**
Char	Nombre correspondant à un caractère ASCII codé	0	255($2^8 - 1$)	**1**
ShortString	Chaine de caractères avec une longueur maximale de 255 caractères			**256**
AnsiString	Chaine de caractères avec une longueur illimitée			**8**
String	Chaine de caractères	Peut-être ShortString ou AnsiString		**256**
String[y]	Chaine de caractères devant pas excéder y caractères			
Pointeur	Adresse mémoire			**4**

